

XP y Scrum: complementarios, no competencia

La pregunta correcta

La pregunta no es “¿usamos XP o Scrum?” sino “¿cómo combinamos Scrum y XP para construir software de calidad con un proceso organizado?”. La comparación entre metodologías como si fueran alternativas excluyentes es un error de categoría: Scrum y XP operan en niveles distintos del trabajo de desarrollo.

La frase del docente resume la relación: “Scrum gestiona el proceso, XP gestiona la técnica.”

Qué hace cada uno

Scrum es un marco de proceso. Define quién hace qué, cuándo y en qué orden. Responde a preguntas de organización del trabajo:

- ¿Quién decide qué se construye? El Product Owner.
- ¿Cómo se planifica el trabajo? Sprint Planning cada dos semanas.
- ¿Cómo se comunica el equipo diariamente? Daily Scrum.
- ¿Cómo se valida el incremento? Sprint Review.
- ¿Cómo mejora el equipo? Retrospectiva.

Scrum no dice nada sobre cómo se escribe el código, cómo se prueba, cómo se integra ni cómo se mantiene su calidad técnica. Esas decisiones las deja al equipo.

XP es un conjunto de prácticas de ingeniería técnica. Define cómo se construye el código con calidad. Responde a preguntas técnicas:

- ¿Cómo se garantiza que el código hace lo que debe? TDD.
- ¿Cómo se detectan problemas de integración temprano? Continuous Integration.
- ¿Cómo se transfiere conocimiento técnico? Pair Programming y Collective Ownership.
- ¿Cómo se mantiene la estructura interna del código? Refactoring.
- ¿Cómo se evita construir complejidad innecesaria? Simple Design.

XP no dice nada sobre cómo se organiza el equipo, cómo se toman decisiones de priorización ni cómo se comunica el progreso a los stakeholders. Esas decisiones las deja a Scrum.

Tabla comparativa

Dimensión	Scrum	XP
Nivel de operación	Proceso	Técnica de ingeniería
Pregunta que responde	¿Quién hace qué, cuándo?	¿Cómo se construye con calidad?

Dimensión	Scrum	XP
Unidad de tiempo	Sprint (2 semanas)	Ciclo de prueba (minutos), build (horas)
Artefactos	Product Backlog, Sprint Backlog, Incremento	Pruebas unitarias, pipeline CI, código refactorizado
Roles	Product Owner, Scrum Master, Developers	Cliente, Programadores, Coach, Tester, Tracker
Ceremonias	Planning, Daily, Review, Retrospectiva	Pair Programming (continuo), Code Review (por historia)
Calidad	Define “potencialmente entregable” (DoD)	Define cómo lograr esa calidad técnicamente
Visibilidad	Tablero, burndown, velocidad	Cobertura de pruebas, defectos por sprint, deuda técnica

Cómo se combinan en este proyecto

En este proyecto, Scrum es el marco de proceso y XP es la capa técnica que opera dentro de los sprints.

Scrum organiza el trabajo:

- El PO (Josué Say) mantiene y prioriza el Product Backlog.
- El SM (Mathew Cordero) facilita las ceremonias y remueve impedimentos.
- Los Developers (Cruz, Chen, Guzmán) se autoorganizan para completar el Sprint Backlog.
- Los 8 sprints de dos semanas estructuran la Fase 2 (enero–abril 2027).

XP define cómo se construye el código dentro de cada sprint:

- TDD se aplica en las historias de EP02, EP04 y EP05.
- CI está activo desde el sprint 1 (HT-01) y verifica cada push al repositorio.
- Pair Programming se aplica en las historias de mayor criticidad (EP04-US01, EP05-US01, módulos de seguridad).
- Refactoring es parte del DoD, no una tarea separada.
- Simple Design evita que el motor de recomendación se sobrediseñe.
- Collective Ownership asegura que los tres developers tienen contexto sobre todo el código base.

La metodología de proceso es Scrum. XP opera como capa técnica dentro de los sprints de Scrum. No hay tensión entre los dos: cada uno responde preguntas que el otro no responde.

Beneficios de la combinación vs Scrum solo

Problema con Scrum solo	Cómo XP lo resuelve
Scrum entrega incrementos “potencialmente entregables” pero no define cómo lograr esa calidad	TDD, CI y Refactoring garantizan la calidad técnica del incremento
El proceso puede ser ordenado mientras el código se deteriora	CI y Code Review hacen visible la calidad técnica sprint a sprint
El conocimiento técnico puede concentrarse en un developer	Pair Programming y Collective Ownership distribuyen el conocimiento
La deuda técnica no es visible en el tablero de Scrum	Las historias técnicas (HT-01 a HT-07) y la métrica de deuda documentada hacen la deuda visible
Los defectos se descubren en el Sprint Review o en producción	TDD y CI detectan defectos en minutos u horas, no en semanas
La integración se prueba solo al final	CI verifica la integración en cada push

Historia documentada: “La calidad no es solo visual”

El docente documenta un patrón recurrente en proyectos que adoptan Scrum sin prácticas XP: el equipo de QA aprobaba módulos visualmente en el Sprint Review sin verificar la funcionalidad real.

La forma en que ocurrió: el revisor de QA accedía a la interfaz del módulo, verificaba que la pantalla cargaba correctamente, que los botones respondían y que los datos se mostraban. El módulo avanzaba a Done. Las pruebas funcionales reales —¿los datos persisten correctamente en la base de datos? ¿El módulo funciona bajo condiciones de error? ¿El comportamiento es correcto con perfiles de datos reales y no solo los usados en la demo?— no se hacían porque no había pruebas automatizadas que las verificaran ni CI que las ejecutara.

El resultado fue un conjunto de módulos que “funcionaban visualmente” pero que tenían defectos de integración, datos que no persistían correctamente en casos no demostrados y comportamiento incorrecto con datos reales que solo aparecieron cuando el sistema se usó en producción.

La frase “La calidad no es solo visual” sintetiza la lección: una demo de dos minutos en el Sprint Review puede mostrar que la pantalla hace lo que promete, pero no puede mostrar que los datos se guardan correctamente, que los errores se manejan bien, que las pruebas automatizadas pasan o que el código puede modificarse sin riesgo. Esos criterios requieren TDD, CI y un DoD que los incluya explícitamente.

En este proyecto, el DoD extendido con criterios XP (XP-1, XP-2, XP-3) y la columna Code Review en el tablero son los mecanismos que previenen exactamente ese patrón. El Sprint Review del proyecto no es el primer punto donde se verifica la calidad: es el punto de validación funcional final, después de que TDD, CI y Code Review ya han verificado la calidad técnica.